Artificial Intelligence Assignment

Title: Smart E-Commerce Recommendation System using AI Pipeline

Task 1: AI Pipeline Design

Data Collection

The system collects user interactions such as purchases and clicks, along with product categories (Electronics, Gadgets, Fitness) to understand preferences.

Preference Learning

Purchases are given higher weight than clicks to better reflect user interest.

Recommendation Process

An affinity profile is created from user history and used to score products. Top-scoring products are recommended.

AI Pipeline Flow

User Interactions
↓
Data Collection
↓
Feature Extraction
↓
Affinity Profile
↓
Recommendation Engine
↓
Top Products

Task 2: User Affinity Profile

User purchased Product 103 and clicked Products 104, 105, 106. Purchased items have higher weight.

Final Affinity Profile:

[2, 2, 5]

This shows strongest interest in Fitness, followed by Electronics and Gadgets.

Code

[1] ✓ 0s

```python
import numpy as np

products = {
    101: [1, 1, 0], 102: [1, 1, 0], 103: [0, 0, 1],
    104: [1, 1, 1], 105: [0, 0, 1], 106: [1, 1, 0], 107: [0, 0, 1]
}
purchased_ids = [103]
clicked_ids = [104, 105, 106]

affinity = np.array([0, 0, 0])
for pid in purchased_ids:
    affinity += np.array(products[pid]) * 3
for pid in clicked_ids:
    affinity += np.array(products[pid])

print("Affinity Profile =", affinity)
```

```
Affinity Profile = [2 2 5]
```

Task 3: Recommendation Engine

Products are scored by comparing features with the affinity profile. Already purchased items are excluded.

Top Results

Product	Score
104	9
105	5
107	5

Recommendations

- Product 104
- Product 105
- Product 107

Code

[2] ✓ 0s

```python
scores = {}
for pid, feature in products.items():
    if pid not in purchased_ids:
        scores[pid] = np.dot(feature, affinity)

top = sorted(scores.items(), key=lambda x: x[1], reverse=True)

print("Top 3 Recommendations:")
for p in top[:3]:
    print(p)
```

```
Top 3 Recommendations:
(104, np.int64(9))
(105, np.int64(5))
(107, np.int64(5))
```

Task 4: Limited Memory Extension

A Limited Memory Agent uses recent interactions to update preferences dynamically.

Interaction Flow

[106, 104, 105]

Memory Updates

Step 1: [106]

Step 2: [106, 104] → [2, 2, 1]

Step 3: [104, 105] → [1, 1, 2]

This shows how preferences change over time.

Code

[3] ✓ 0s

```
memory = []
sequence = [106, 104, 105]

for pid in sequence:
    memory.append(pid)
    if len(memory) > 2:
        memory.pop(0)

    score = np.array([0, 0, 0])
    for m in memory:
        score += np.array(products[m])

    categories = ["Electronics", "Gadgets", "Fitness"]
    print("Memory:", memory)
    print("Current Interest:", categories[np.argmax(score)])
```

```
Memory: [106]
Current Interest: Electronics
Memory: [106, 104]
Current Interest: Electronics
Memory: [104, 105]
Current Interest: Fitness
```

Task 5: Critical Analysis

1. Why Limited Memory Agent?

It improves personalization using past interactions, unlike reactive systems.

2. Changing Interests

The system updates preferences based on recent behavior.

3. Improvements

- Collaborative Filtering
- Deep Learning Models

4. AI Type

This is **Narrow AI** (task-specific system).

5. Limitation

Fixed weights cannot fully capture complex behavior.

Bonus

Recent interactions should have higher weight for better accuracy.

Conclusion

The system builds an affinity profile from user behavior and recommends relevant products effectively.

Final Profile: [2, 2, 5]

Top Products:

- Product 104
- Product 105
- Product 107